

Análise do desempenho de aplicações *CPU-bound* entre *P-Core* e *E-Cores*

JOÃO LUIZ GRAVE GROSS, Instituto de Informática da UFRGS, Brasil

A consolidação de arquiteturas de processadores híbridos, combinando núcleos de desempenho (*P-Cores*) e de eficiência (*E-Cores*), impõe novos desafios para a otimização de *software*, especialmente em linguagens com limitações de concorrência como *Python*. Este trabalho avalia o impacto dessa heterogeneidade de *hardware* no desempenho de aplicações *CPU-bound*, utilizando a multiplicação de matrizes como estudo de caso. Foram comparadas três estratégias de paralelismo: a biblioteca padrão *Multiprocessing*, o escalonador dinâmico *Dask* e o modo experimental *Free-Threading* (Python 3.14), sob diferentes configurações de afinidade de núcleo e tamanhos de carga. Os experimentos demonstraram que, em cargas de trabalho intensivas, o uso indiscriminado de *E-Cores* atua como gargalo para o *Multiprocessing*, enquanto o *Dask* mitiga esse efeito através de escalonamento dinâmico. Contudo, a implementação *Free-Threading* apresentou superioridade absoluta, entregando os menores tempos de execução e maior eficiência energética, com temperatura média de operação cerca de 30°C inferior às demais abordagens. Conclui-se que a remoção do *Global Interpreter Lock* (GIL) aliada a uma gestão consciente da topologia da CPU são fundamentais para maximizar o desempenho em arquiteturas modernas.

Palavras-Chave: P-Core, E-Core, Hyper-Threading, Intel Turbo Boost, Afinidade de CPU, Python GIL, Multiprocessing, Dask, Threading

Formato de Referência da ACM:

João Luiz Grave Gross. 2025. Análise do desempenho de aplicações *CPU-bound* entre *P-Core* e *E-Cores*. Disciplina: Computer System Performance Analysis (CMP223). (Novembro 2025), 9 páginas.

1 Introdução

A indústria de processadores passou recentemente por uma mudança de paradigma com a consolidação de arquiteturas híbridas, como a *Intel Hybrid Technology* [3]. Esse modelo abandona a homogeneidade tradicional para integrar, no mesmo encapsulamento, núcleos de alta performance (*P-Cores*), projetados para tarefas críticas e de baixa latência, e núcleos de eficiência (*E-Cores*), otimizados para tarefas de fundo e economia energética. Embora essa heterogeneidade maximize a relação desempenho/*watt*, ela introduz uma nova camada de complexidade para o desenvolvimento de *software*, exigindo que algoritmos e sistemas operacionais orquestruem corretamente a alocação de tarefas para evitar gargalos de desempenho decorrentes do uso inadequado dos núcleos.

Nesse contexto, este trabalho utiliza a multiplicação de matrizes como carga de trabalho estritamente limitada pela CPU (*CPU-bound*) para avaliar o comportamento dessa arquitetura. Devido à sua natureza altamente paralelizável e ao uso intensivo de unidades de ponto flutuante, esse cenário permite estressar tanto *P-Cores* quanto *E-Cores*, além de investigar a influência do *Hyper-Threading* no desempenho computacional. A análise compara como estratégias consolidadas em Python (*Multiprocessing* e *Dask*) e a nova abordagem *Free-Threading*, viabilizada pela remoção da trava do *Global Interpreter Lock* (GIL) na versão 3.14.0 do interpretador, gerenciam a distribuição de carga e o tempo de execução frente à assimetria e aos recursos de concorrência do *hardware*.

2 Plataforma: Hardware e Sistema Operacional (SO)

A infraestrutura computacional utilizada nos experimentos consiste em uma estação de trabalho Dell OptiPlex 7020 Micro. As especificações de *hardware* incluem uma CPU Intel Core i7-14700T (14ª Geração, 20 cores, sendo 8 *P-Cores*/16 *threads* e 12 *E-Cores*/12 *threads*, cache de 33 MB, 1.3GHz a 5.0GHz), 16GB de RAM DDR5 5600MT/s (módulo único) e SSD de 512GB NVMe M.2 [1]. O ambiente de *software* baseia-se no sistema operacional Ubuntu 24.04 LTS [5].

A seleção desta plataforma baseou-se na disponibilidade de acesso exclusivo e irrestrito (24/7), permitindo a execução dos experimentos em um ambiente isolado, livre da contenção de recursos típica de sistemas compartilhados. Fisicamente, o equipamento encontra-se alocado em um *Data Center* onde a temperatura é mantida estável entre 17°C e 21°C, mitigando oscilações térmicas que poderiam interferir nos resultados. Para assegurar a operação remota completa, o sistema foi configurado com a tecnologia *Intel vPro*, possibilitando o gerenciamento *out-of-band* — incluindo reinicialização, acesso à BIOS e seleção SO via menu GRUB remotos — enquanto o acesso ao sistema operacional foi realizado via protocolo RDP.

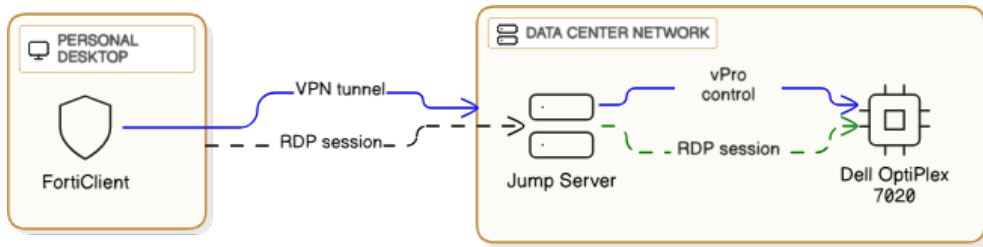


Figura 1. Acesso e controle da plataforma de experimentos

3 Algoritmo de Multiplicação de Matrizes

A multiplicação de matrizes foi selecionada como a carga de trabalho base para a avaliação de desempenho entre *P-Cores* e *E-Cores*. Esta escolha fundamenta-se na natureza estritamente *CPU-bound* do problema e em sua alta capacidade de paralelização, características que permitem mensurar o desempenho computacional do processador e distribuir um conjunto de tarefas entre os múltiplos núcleos.

O algoritmo recebe como parâmetro de entrada a dimensão N e procede para a criação de duas matrizes $N \times N$ inicializadas com dados pseudoaleatórios. Ele então segmenta o problema, criando 8 tarefas distintas, onde cada uma é responsável por computar uma fatia equivalente do cálculo total. Ao término da execução paralela, o sistema realiza a concatenação dos 8 blocos de resultados parciais para compor a matriz final da multiplicação.

4 Características das Implementações

Foram desenvolvidas três implementações para o algoritmo de multiplicação de matrizes. Embora todas se fundamentem no alto nível de paralelismo inerente ao problema, cada uma adota uma estratégia de gerenciamento de memória distinta.

A primeira implementação foi executada sobre o interpretador *Python 3.12.12* com o módulo *multiprocessing*. Nesta abordagem, instanciam-se 8 processos filhos independentes através da *system call* `fork()`. O método beneficia-se do mecanismo de *Copy-on-Write* (CoW), onde o espaço de endereçamento do processo pai é compartilhado virtualmente com os filhos. Visto que as operações

nas matrizes são estritamente de leitura, elimina-se a sobrecarga (*overhead*) de duplicação ou transferência de dados, otimizando o consumo de recursos.

A abordagem com Dask, também para o *Python 3.12.12*, baseia-se no particionamento das matrizes em blocos. Dadas as limitações de configuração explícita de afinidade por *worker* no comando nativo da *.matmul*, a estratégia de controle de CPU é aplicada no nível do processo pai. Utiliza-se o conceito de **Herança de Afinidade**: define-se a afinidade da CPU no processo principal antes da execução do *fork* que cria os *workers*. Por consequência, quando os *workers* são instanciados, o *Kernel* garante que eles herdem as restrições do pai, assegurando que as tarefas distribuídas pelo Dask sejam processadas estritamente pelos núcleos designados.

Por fim, a implementação com *threading*, desenvolvida para o *Python 3.14.0*, explora a remoção do *Global Interpreter Lock* (GIL), recurso nativo desta versão (modo *free-threading*) que viabiliza o paralelismo real entre *threads*. A memória do processo pai é compartilhada entre todas as *threads* através do mecanismo *Zero Copy*, ou seja, diferentemente do CoW, não há duplicação de páginas de memória mesmo em caso de escrita, pois o acesso é direto ao espaço global compartilhado.

Todas as implementações registram, ao término de cada execução, o horário e a temperatura da CPU. Ainda, a fim de assegurar a estabilidade dos resultados, o *governor* do processador foi configurado estritamente no modo *performance*, forçando a operação dos núcleos na frequência máxima de *clock* suportada.

5 Cenários dos Testes

Com o objetivo de avaliar o desempenho dos *P-Cores* e *E-cores* na execução da multiplicação de matrizes, foram projetados quatro cenários de teste. Conforme detalhado na Seção 3, são geradas oito tarefas computacionais simultâneas, portanto, cada cenário é composto por um conjunto distinto de oito núcleos de processamento.

O *hardware* utilizado possui CPU 28 núcleos lógicos com *IDs* variando de 0 a 27. *IDs* de 0 a 15 são de núcleos *P-Core*, de modo que cada par sequencial de *IDs* (0-1, 2-3, ..., 14-15) compartilha os recursos de um mesmo núcleo físico. Já *IDs* de 16 a 27 correspondem aos núcleos *E-Core*, onde cada identificador representa estritamente um núcleo físico.

Abaixo a descrição dos cenários de testes:

8P_HT_OFF: Utiliza 8 *P-Cores* com *Hyper-Threading* desabilitado (1 *thread* por núcleo físico). Os *IDs* utilizados são {0, 2, 4, 6, 8, 10, 12, 14}.

4P_HT_ON: Utiliza 4 *P-Cores* com *Hyper-Threading* habilitado (2 *threads* por núcleo físico), totalizando 8 *threads* lógicas. Os *IDs* utilizados são {0, 1, 2, 3, 4, 5, 6, 7}.

4P_4E_HT_OFF: Configuração híbrida com 4 *P-Cores* e 4 *E-Cores*. Os *IDs* utilizados combinam núcleos de performance {0, 2, 4, 6} e de eficiência {16, 17, 18, 19}.

8E: Utiliza 8 *E-Cores* (1 *thread* por núcleo físico). Os *IDs* lógicos utilizados são {16, 17, 18, 19, 20, 21, 22, 23}.

Em todos os cenários avaliados, optou-se pela manutenção da tecnologia *Intel Turbo Boost* habilitada, permitindo que a frequência de operação atinga os valores máximos permitidos. A desabilitação deste recurso confinaria os núcleos à sua frequência base [4], o que comprometeria os experimentos. Embora reconheça-se que a elevação da temperatura da CPU possa levar à redução forçada da frequência (*thermal throttling*), este risco foi mitigado com intervalos de resfriamento de dez segundos entre as execuções consecutivas.

6 Ambiente e Configurações

A orquestração e execução dos códigos foram realizadas utilizando o ambiente interativo *Jupyter Notebook*. Dado o requisito de avaliação comparativa entre versões distintas do interpretador,

empregou-se a ferramenta `pyenv` para o gerenciamento de versões. Foram instaladas e configuradas as versões *Python 3.12.12* (estável) e *Python 3.14.0t* (experimental *free-threading*). Previamente à inicialização do servidor *Jupyter*, a versão alvo foi selecionada explicitamente com os comandos `pyenv local {py_version}` e `pyenv shell {py_version}`.

Também foi necessário neutralizar o paralelismo implícito de baixo nível presente nas bibliotecas de álgebra linear (BLAS/LAPACK) utilizadas pelo *NumPy*. Para garantir que a operação vetorial `np.dot()` fosse executada de maneira serial (*single-threaded*) dentro de cada processo da aplicação, as seguintes variáveis de ambiente foram configuradas com valor 1:

```
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 MKL_NUM_THREADS=1
VECLIB_MAXIMUM_THREADS=1 NUMEXPR_NUM_THREADS=1
```

Além da seleção da versão adequada do interpretador e da parametrização para o *Numpy*, a utilização da versão *Python 3.14.0t* requer a desativação explícita do GIL via variável de ambiente. Assim, as instruções de linha de comando para a inicialização do servidor *Jupyter*, referentes aos ambientes 3.12.12 e 3.14.0t, apresentam-se, respectivamente, da seguinte forma:

Para *Python 3.12.12*: `OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 VECLIB_MAXIMUM_THREADS=1 OPENBLAS_NUM_THREADS=1 NUMEXPR_NUM_THREADS=1 jupyter notebook`

Para *Python 3.14.0t*: `OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 VECLIB_MAXIMUM_THREADS=1 OPENBLAS_NUM_THREADS=1 NUMEXPR_NUM_THREADS=1 PYTHON_GIL=0 jupyter notebook`

7 Projeto Experimental

Para criar o projeto experimental, foram definidos 5 tamanhos de matriz de entrada: 1000, 2000, 4000, 6000 e 8000. Dimensões superiores a 8000 foram desconsideradas devido às restrições de memória da plataforma (evitando estouro de memória). O experimento avaliou 4 cenários distintos de alocação de núcleos de CPU. Para cada combinação de cenário $\times$ tamanho de matriz, foram realizadas 30 execuções, visando aproximar a distribuição dos dados de uma normal (Teorema do Limite Central) e assegurar um intervalo de confiança robusto.

As 600 configurações resultantes (4 cenários $\times$ 5 tamanhos $\times$ 30 repetições) foram geradas via script e a sua ordem foi aleatorizada. Esse embaralhamento é crucial para mitigar correlações temporais ou vieses entre as execuções. Adicionalmente, inseriu-se uma etapa de *warm-up* no início da bateria de testes, composta por 6 execuções com matriz de tamanho 8000 e oito *P-Cores* de *thread* única. O objetivo desta etapa é permitir que a CPU atinja um estado estável de temperatura, sendo os resultados destas execuções descartados posteriormente.

Por fim, para garantir a reprodutibilidade e a igualdade da comparação, as três implementações fixaram a mesma semente (`np.random.seed(42)`). Isso assegura que todas as implementações processem exatamente as mesmas matrizes de entrada.

O resultado final é um arquivo `.csv` contendo 606 configurações: as 6 primeiras de *warm-up* e as 600 restantes pertencentes ao experimento.

8 Análise dos Resultados

Esta seção destina-se à análise dos resultados obtidos ao executar o projeto experimental nas três implementações.

8.1 Evolução da Temperatura

Como ponto de partida para a avaliação de desempenho, examinou-se o comportamento térmico da CPU à medida que as execuções são realizadas.

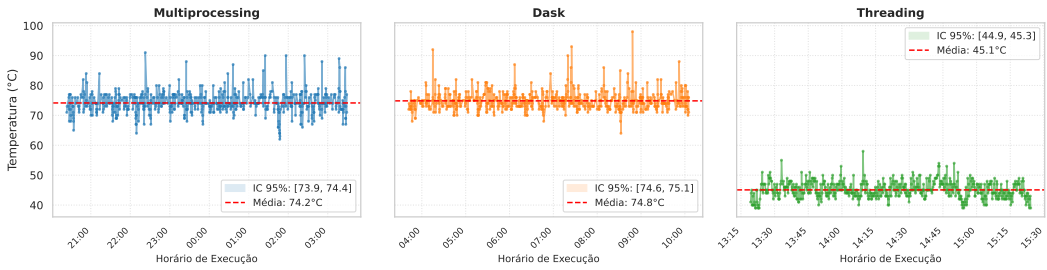


Figura 2. Evolução térmica por implementação (Multiprocessing, Dask, Threading)

A Figura 2 já incorpora o incremento térmico gerado pela execução inicial das seis cargas de trabalho utilizadas para o *warm-up*. Observa-se que as estratégias baseadas em *Multiprocessing* e *Dask* alcançaram temperaturas médias de 74.2°C e 74.8°C, respectivamente. Esses valores refletem um uso intensivo dos núcleos de processamento, associado ao custo de ativar e manter múltiplos processos simultâneos, cada qual com seu próprio contexto de execução. Em contrapartida, a implementação com *Threading* apresentou temperatura média significativamente inferior, de 45.1°C.

Essa diferença evidencia que o modelo baseado em *threads* impõe menor demanda à CPU. Tal comportamento decorre, possivelmente, das limitações de concorrência introduzidas pelo GIL, que restringe a execução paralela efetiva de trechos de código em Python, reduzindo o nível de competição entre unidades de processamento e, consequentemente, a dissipação de calor. Além disso, a política de escalonamento do *kernel* tende a distribuir *threads* de maneira mais balanceada entre os núcleos disponíveis, o que reduz picos de carga característicos de abordagens que utilizam múltiplos processos independentes. Esses fatores combinados explicam a menor elevação térmica observada na estratégia de *Threading*.

8.2 Tempo de execução por cenário

Inicialmente, realizou-se a análise dos tempos médios de execução para os quatro cenários considerando matrizes de tamanho 1000 e 2000. Essa segmentação foi necessária pois, a partir do tamanho 4000, observa-se um aumento expressivo na ordem de grandeza dos tempos, o que prejudicaria a escala visual e a comparação detalhada dos casos menores.

Ao analisar a Figura 3, observa-se o comportamento dos três scripts nos quatro cenários propostos. A abordagem de *Threading* demonstrou desempenho superior, apresentando os menores tempos médios em todos os casos.

Na comparação entre *Multiprocessing* e *Dask* para matrizes de tamanho 1000 e 2000, nota-se um comportamento similar. O *Multiprocessing* mostrou-se ligeiramente mais eficiente, sem sobreposição dos intervalos de confiança de 95% em relação ao *Dask*, o que indica que *Multiprocessing* é estatisticamente superior nestes casos.

Entretanto, a análise da Figura 4 revela uma inversão de tendência. Para matrizes de tamanho 4000 e 6000, o *Dask* supera o desempenho do *Multiprocessing* em quase todos os cenários (exceto em 4P_HT_ON).

Esse comportamento pode ser explicado pela forma como ocorre a gestão de recursos. Enquanto o *Multiprocessing* pode ser forçado a utilizar todos os núcleos disponíveis (incluindo os *E-cores* de menor desempenho), o *Dask* gerencia um *pool* de recursos onde seu *Scheduler* aloca tarefas dinamicamente. Essa flexibilidade permite que o *Dask* priorize o uso de *P-Cores*, resultando em ganho de performance, comportamento especialmente visível no cenário 4P_4E_HT_OFF.

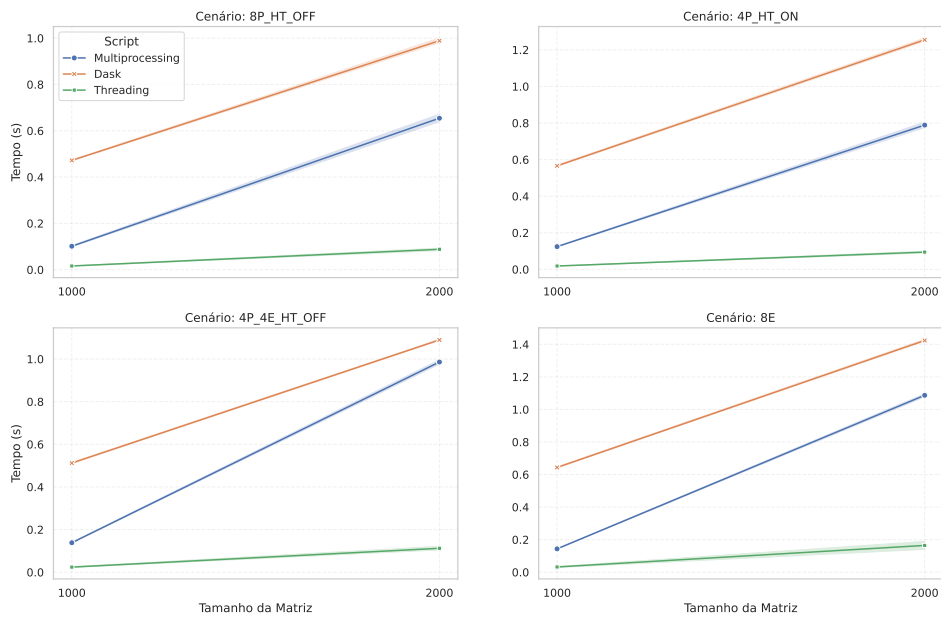


Figura 3. Tempo Médio de Execução (Matrizes 1000 e 2000) - I.C. 95%

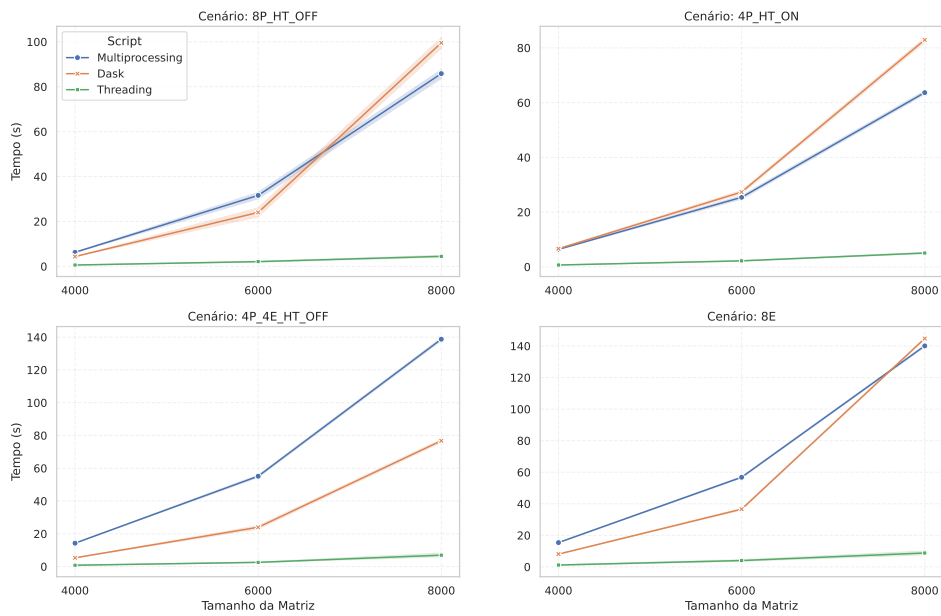


Figura 4. Tempo Médio de Execução (Matrizes 4000, 6000 e 8000) - I.C. 95%

8.3 Tempo de execução por tamanho de matriz

A Figura 5 detalha os tempos médios de execução para os três *scripts*, segmentados pelos cinco tamanhos de matriz analisados. Reafirma-se o comportamento observado anteriormente, onde o *Dask* apresenta tempo de execução médio superior ao *Multiprocessing* e *Threading* para cargas menores (1000 e 2000). Embora o desempenho entre os cenários seja similar, nota-se uma elevação nos tempos totais quando o *Hyper-Threading* está ativo (4P_HT_ON) ou quando a execução ocorre exclusivamente em *E-Cores* (8E).

Tal comportamento é esperado, pois, no caso do *Hyper-Threading*, ocorre o compartilhamento dos recursos do núcleo físico (como cache e unidades de execução) entre dois fluxos de processamento, gerando contenção que não existe na execução *single-thread* por núcleo. Já no cenário de *E-Cores*, o aumento do tempo justifica-se pelas especificações de hardware, uma vez que esses núcleos operam em frequências menores e possuem arquitetura voltada para eficiência energética, não para alto desempenho.

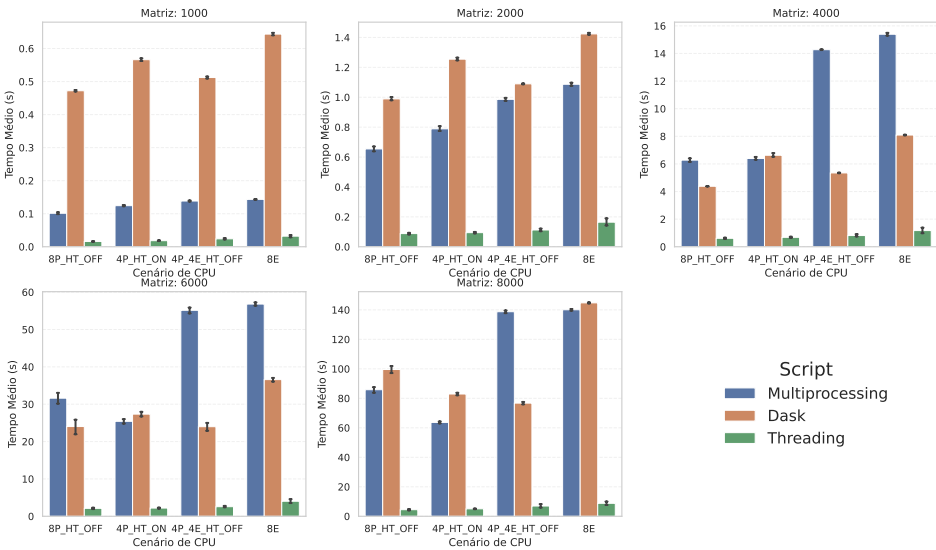


Figura 5. Tempo Médio de Execução por Tamanho de Matriz - I.C. 95%

Adicionalmente, observa-se que, para matrizes com dimensão superior a 4000, a abordagem de *Multiprocessing* sofre severa degradação de desempenho ao utilizar *E-Cores*. Com o aumento da carga computacional, o sistema operacional aloca processos nesses núcleos de menor capacidade, criando um gargalo. Este efeito é mitigado no *Dask*, cujo escalonador dinâmico gerencia um *pool* de recursos e prioriza a alocação de tarefas nos *P-Cores*, evitando o uso ineficiente dos *E-Cores* quando possível. Contudo, em cenários onde o *pool* é restrito exclusivamente a *E-Cores* (Cenário 8E), o aumento no tempo de execução do *Dask* torna-se inevitável.

8.4 Tempo de execução por script

A Figura 6 destaca o comportamento do sistema para cargas de trabalho intensivas (matriz de tamanho 8000). Nota-se que o cenário 4P_HT_ON (*Hyper-Threading* habilitado) posicionou-se entre os menores tempos médios de execução. Estatisticamente, para o script de *Threading*, os cenários

baseados exclusivamente em *P-Cores* (4P_HT_ON e 8P_HT_OFF) apresentaram desempenho equivalente, por haver sobreposição de intervalos de confiança. O mesmo fenômeno de equivalência estatística ocorreu entre os cenários que envolvem *E-Cores* (4P_4E_HT_OFF e 8E).

Entretanto, há nítida diferença de desempenho entre os grupos com e sem *E-Cores*. À medida que o tamanho da matriz cresce, a execução torna-se estritamente limitada pela CPU (*CPU-bound*). Nessas condições, o *overhead* de criação de processos ou *threads* torna-se irrelevante frente ao tempo total de processamento. Consequentemente, a performance passa a depender diretamente da frequência de *clock* dos núcleos. Como os *E-Cores* operam em frequências inferiores e possuem arquitetura simplificada, eles agem como gargalos em tarefas paralelizadas simetricamente, elevando o tempo médio total quando comparados aos cenários de *P-Cores* puros.

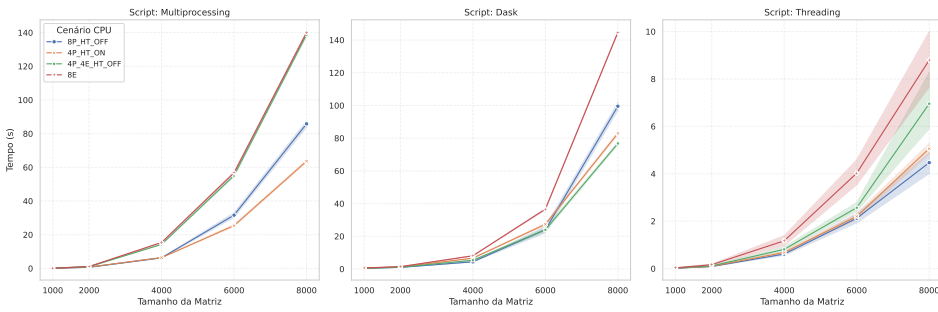


Figura 6. Tempo Médio de Execução por Script (Matriz 8000) - I.C. 95%

9 Reprodutibilidade

Para garantir a reprodutibilidade dos experimentos, todos os scripts desenvolvidos foram disponibilizados publicamente no GitHub [2]. Adicionalmente, as especificações completas dos ambientes de software, incluindo dependências do Python e pacotes do sistema operacional, foram exportadas e anexadas ao repositório.

Os comandos a seguir detalham o processo de exportação das dependências:

```
# 1. Exportar dependências do ambiente Python 3.12
pyenv shell 3.12.12
pip freeze > requirements_py3.12.txt

# 2. Exportar dependências do ambiente Python 3.14 (Free-threaded)
pyenv shell 3.14.0t
pip freeze > requirements_py3.14t.txt

# 3. Exportar pacotes do sistema (Ubuntu) instalados manualmente
apt-mark showmanual > system_packages.txt
```

Para a reconstituição do ambiente experimental, devem ser executadas as seguintes instruções:

```
pip install -r requirements_py3.12.txt
pip install -r requirements_py3.14t.txt
sudo xargs apt-get install -y < system_packages.tx
```


10 Conclusões

Este estudo avaliou o desempenho de três estratégias de paralelismo em *Python* — *Multiprocessing*, *Dask* e *Threading* (modo *free-threading*) — sob a uma arquitetura de processadores híbrida Intel Core de 14ª Geração. A partir dos dados experimentais foi possível extrair inferências determinantes sobre como o *hardware* e as implementações interagem com cargas intensivas em CPU, como a multiplicação de matrizes.

Dentre as implementações a versão *Python 3.14.0t* demonstrou superioridade consistente. Ela apresentou os menores tempos de execução em todos os cenários e tamanhos de matriz, aliando alto desempenho à eficiência energética, evidenciada pela temperatura média de operação significativamente inferior ($\approx 45^{\circ}\text{C}$ contra $\approx 74^{\circ}\text{C}$ das demais). Isso sugere que a remoção do GIL e a utilização de memória compartilhada (*Zero Copy*) eliminam gargalos críticos de serialização e *overhead* de comunicação, tornando-se uma abordagem promissora para computação paralela em *Python* no futuro próximo.

Comparando *Multiprocessing* e *Dask*, observou-se um *trade-off* claro. O *Multiprocessing* é mais eficiente para matrizes menores (até 2000), onde o *overhead* de gestão do *Dask* penaliza o desempenho. Contudo, em cenários de *hardware* heterogêneo (mistura de *P-Cores* e *E-Cores*) com grandes volumes de dados, o *Dask* superou o *Multiprocessing*. A inteligência do escalonador do *Dask* permitiu evitar, quando possível, a alocação de tarefas críticas em núcleos de eficiência, enquanto o *Multiprocessing*, dependente do escalonador genérico do SO, sofreu degradação de performance ao distribuir tarefas pesadas para os *E-Cores*.

Assim, no que se refere à arquitetura de *hardware* (*P-Cores* vs. *E-Cores*), a heterogeneidade impõe desafios para algoritmos de divisão simétrica de tarefas. Em cargas de trabalho estritamente *CPU-bound*, os *E-Cores* atuam como gargalos. Quando uma tarefa é dividida igualmente entre núcleos de performance e eficiência, o tempo total de execução é limitado pelo núcleo mais lento. Os experimentos mostraram que o uso de *Hyper-Threading* nos *P-Cores* oferece desempenho competitivo e estatisticamente equivalente ao uso de núcleos físicos dedicados em certas configurações, enquanto o uso exclusivo de *E-Cores* resulta invariavelmente nos piores tempos devido à menor frequência de operação e arquitetura simplificada.

Por fim, o tamanho de matriz apresentou impacto direto nos experimentos. Para entradas pequenas, o tempo de execução médio foi dominado pelo custo de criação e gerenciamento de processos/*threads*. À medida que a matriz cresce (acima de 4000), o problema torna-se puramente computacional. A partir deste momento a eficiência do código é fortemente dependente da frequência bruta da CPU, destacando as limitações dos *E-Cores* e a vantagem da arquitetura de memória compartilhada do *free-threading*.

Em suma, para maximizar o desempenho em arquiteturas híbridas, deve-se priorizar implementações que evitem a cópia excessiva de dados e, idealmente, que possuam consciência da topologia do processador para evitar a subutilização de recursos causada pela diferença de potência entre *P-Cores* e *E-Cores*.

References

- [1] Dell. 2025. Desktop OptiPlex Micro. <https://www.dell.com/pt-br/shop/cty/pdp/spd/optiplex-7020-micro>.
- [2] João Luiz Grave Gross. 2025. Trabalho Final CMP223. <https://github.com/jlggross/TF-cmp223>.
- [3] Intel. 2021. Arquitetura híbrida de alto desempenho. <https://www.intel.com/content/www/us/en/developer/articles/technical/hybrid-architecture.html>.
- [4] Intel. 2025. How Intel Technologies Boost Your CPU's Performance. <https://www.intel.com/content/www/us/en/gaming/resources/how-intel-technologies-boost-cpu-performance.html>.
- [5] Ubuntu. 2025. ubuntu 24.03 LTS. <https://ubuntu.com/download/desktop>.